

Examen SO

- Cursuri SO - RD
  - Curs 1 - introducere

Stiva Software
app
Framewords/ Middleware
Libraries
Kernel / Drivere
Hardware

- Avem aceste straturi ale stivei ca sa putem modifica individual fiecare strat fara a altera un altul
- Avem o impartire pe verticala, impartita pe rsponsabilitati, intrucat straturile inferioare ofera servicii straturilor superioare
  - Hardware-ul expune ISA (Instruction Set Architecture) {instructiuni, registre} pt soft
- Sistemul de operare ofera portabilitate intre diferite componente hardware

- Avem nevoie de libraries (portabilitate fata de sistemul de operare - intre diferite sisteme de operare) deoarece ne ofera mai multe functionalitati
- Beneficiul unei app dezvoltate direct pe hardware (bare metal) este viteza
- overhead → atunci cand un element iti reduce performanta = costul sau resursele necesare pentru a realiza o sarcina, dar care nu aduce valoare produsului final. Totodata se poate referi la memoria aditionala utilizata necesara pentru a gestiona operatiile si care au efect asupra performantei si eficientei.
- Sistemul de operare este necesar deoarece este mai mare cererea decat ce poate oferi hardwareul (are rol de mediere). Adica exista mai multe procese care au nevoie de resurse. (Daca am avea o singura aplicatie, nu ar fi nevoie de sistem de operare deoarce ar rula doar el pe procesor)
- Bibliotecile sunt generice, iar apps pot linka aceste biblioteci pentru a crea noi functionalitati specifice.
- Sistemul de operare trebuie sa fie izolat de restul aplicatiilor ca sa nu poata fi ‘busit’ de acestea. Astfel, izolarea este mai importanta decat portabilitatea deoarece altfel so ar fi vulnerabil si ar putea fi linkat cu orice aplicatie.
- Izolam so prin existenta a 2 niveluri: kernel/ supervisior mode(controlul memoriei, control io - cand o aplicatie are nevoie sa comunice cu tastatura trece prin kernel mode) si user/application mode.
- Aplicatiile ruleaza direct pe hardware, insa unele tranziteaza zona de kernel pentru operatii critice
- Dezavantajul apelurilor de sistem ⇒ overhead!
- Orcine poate face un apel de sistem, dar nu oricine poate beneficia (userul primeste erori de permisiuni)

▼ Curs 2 - Apeluri de sistem

- strcpy nu face apel de sistem - copierea din memorie nu este privilegiata
- write este un wrapper peste apelul de sistem write → doar se pun params dati functiei write pe registri si se apeleaza functia(apelul de sistem)

- **Rolurile so**: portabilitate (poate rula pe mai multe procesoare - deviceuri diferite), izolare (separare, compartimantare partitionare sau planificare - mai multep procese pot avea acces la resurse fara a se suprapune) si serviciu software (precum o biblioteca - aloci memorie, comunici cu io, creezi procese, sincronizare procese, creare sisteme de fisiere, asigura comunicarea intre procere ⇒ interfata numita SYSCALL API)

▼ Interfata cu sistemul de fisiere

- IO reprezinta un serviciu oferit de kernel mode pentru a oferi portabilitate (ca sa fim siguri ca nu se incurca cu datele altui proces)
- Un device are nevoie de un protocol de comunictie, un canal, interfata si adresa pentru a comunica cu un alt IO
- IO interface pt a crea o interfata generica de comunicare a unui device cu alte device uri

> 💡 ADRESA/NUME ⇒ HANDLE ⇒ OPERATII (pentru a face operatii pe un fisier)

▼ De ce am nevoie de fisiere?

- Ca sa putem stoca datele unei aplicatii (avem nevoie de persistenta acestora)
- Ca sa putem comunica intre aplicatii

- Operatii pe IO: open, read, write, close, seek(pozitioanare in cadrul fisierului), create, delete(stergi entitatea), trunchiere(du-te pana la mijloc si sterge de acolo). Diferenta dintre operatiile create, delete si celelate operatii este data de faptul ca delete si create primesc o adresa, iar celelalte un file descriptor(id).
- Apelul de sistem “open” intoarce file descriptorul (primul disponibil dintr-un vector: FD table)

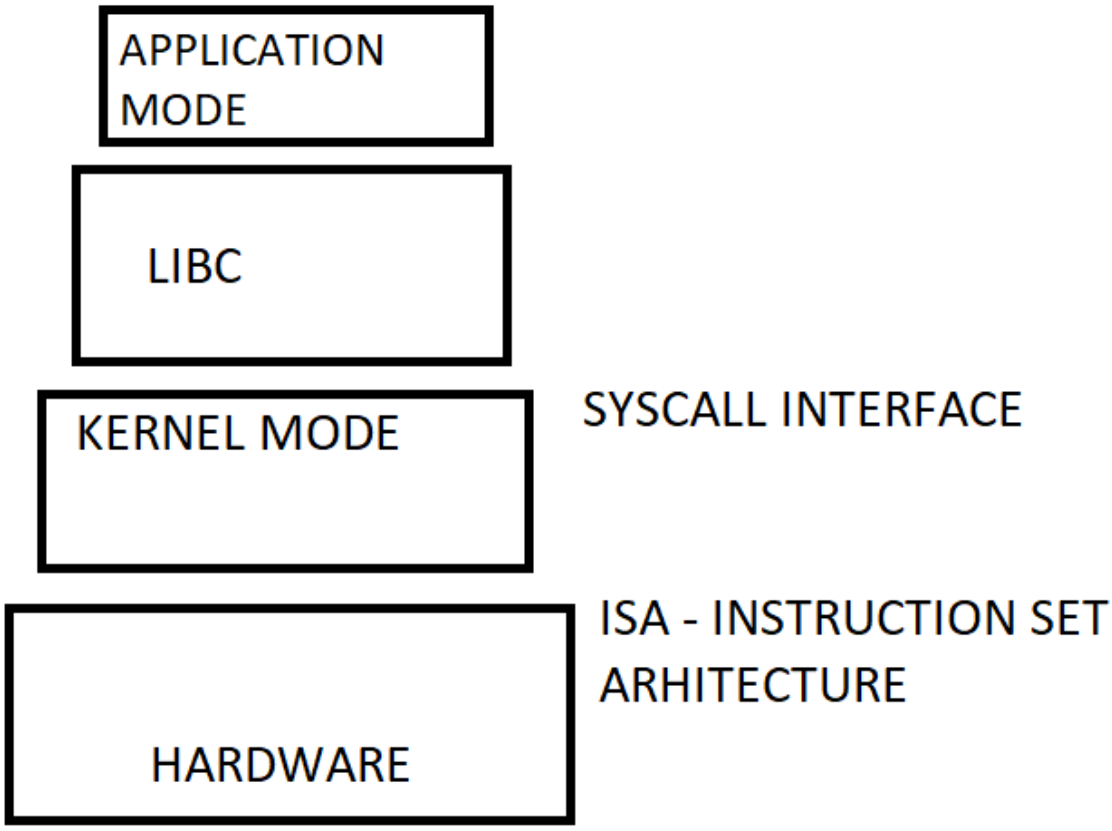
File descriptor table
0 → stdin

1 → stdout
2 → stderr
3 → primul disponibil

- Fiecare proces are asociat un file descriptor table(tabela de fisiere), aceasta avand dimensiunea de 1024 octeti(este limitat din motive de securitate → atacuri de tip denial of service).

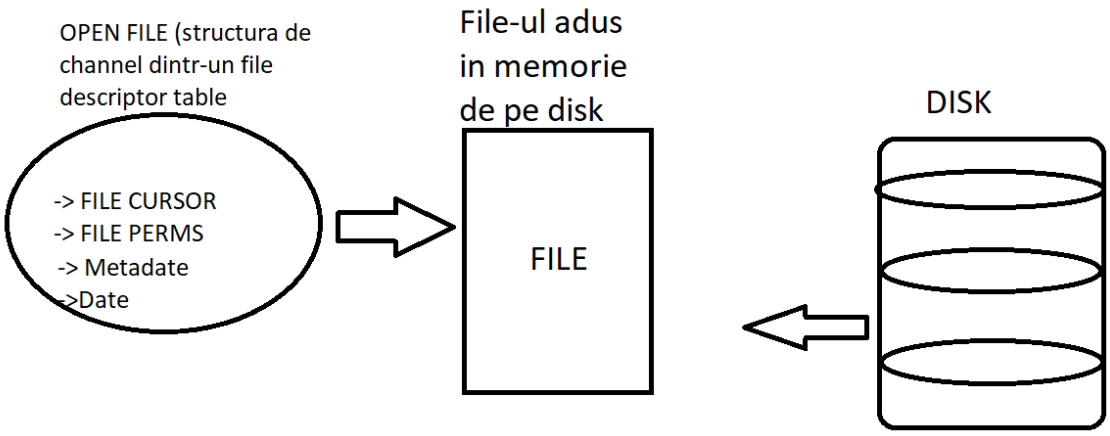
▼ Curs 3 - Interfata sistemului de fisiere

- Interfata expusa de hardware - ISA
- Interfata expusa de kernel mode - syscall interface



- Lucrul cu stringurile nu genereaza apel de sistem deoarece doar se acceseaza memoria. In schimb strdup face apel de sistem deoarece aloca memorie.

- Un apel de sistem cauzeaza mai mult overhead decat unul din biblioteca din cauza tranzitiei dintre user mode in kernel mode(se schimba modul procesorului, registri)
- Putem avea un apel de biblioteca care sa genereze: mai multe apeluri de sistem, un apel de sistem sau niciun apel de sistem
- Putem avea un apel de sistem generat: de mai multe apeluri de biblioteca(fopen si open genereaza acelasi apel de sistem), de un singur apel de biblioteca(cred ca epoll) sau syscall care nu e generat de niciun apel de biblioteca
- Asemanari intre libs si so: ofera un API, amandoua contin cod obiect (direct compilat si doar il folosesc), sunt independente de aplicatie.
- Diferente intre libs si so: sistemul de operare nu se linkeaza, bibliotecile necesita so ca sa functioneze, sistemul de operare se gaseste in kernel mode, iar libs in user mode
- Tabela de file descriptori contine pointeri catre structuri de canal (catre un fisier pe disk, catre utilizator, tastatura sau monitor, catre socket)
- In tabela de file descriptori putem avea referinta catre un fisier deschis care este adus de pe disk in memorie (din motive de performanta, disk-ul fiind mai incet decat memoria RAM). Initial de aduc metadatele, iar apoi se aduc la cerere si datele (on demand)



- FILE CURSOR poate fi diferit in functie de modul in care deschidem fisierul: poate fi 0 sau dimensiunea file-ului. Acesta poate fi modificat prin operatii

precum: write, read, lseek

- Intr-o aplicatie grafica, primii 3 file descriptori nu mai sunt pt stdin, stdout, stderr ci dev/null (nu e folosit) si cum nu se scrie catre un terminal, avem pointer catre un fisier (“startup.log” - teams) pt scriere si citire
- 2 FDT pot avea un file descriptor care sa pointeze catre aceleasi metadate in acelasi timp ⇒ in teorie 2 procese pot scrie in acelasi timp intr-un fisier
- Apelul de sistem write se face pe datele aduse de pe disk in memorie, iar pentru a le trimite din nou pe disk se face sync (cand se inchide fisierul) sau flush (avantajul unui sync mai des este fiabilitatea - risc mai mic de a ne pierde datele daca se ia curentul, iar dezavantajul ar fi cauzat de un overhead mai mare)
- Exista un apel de sistem “dup” care are rolul de a duplica file descriptorii ⇒ pot exista 2 intrari in tabela care sa pointeze catre acelasi channel structure
- Structura de canal retine si un ref count ce retine numarul de referinte catre file. Daca exista doar o referinta, atunci cand se da close de sterge file descriptorul din tabela si se dealoca memoria pt channel.

Operatie	**Cursor**	**Dimensiune file**
open	initializare (0 sau dim file)	daca se deschide cu permisiune de write se suprascrie file-ul deci se schimba dimensiunea
seek	il muta oriunde (inclusiv mai departe de dimensiunea file-ului)	nu
read	da (citesc byte cu byte)	nu
write	da (in sens pozitiv)	da, cand ajunge aproape de limita si se creste dimensiunea fisierului
truncate	nu	da, cum vreau eu

- Daca facem truncate in jos, se pierd datele, dar daca facem in sus e ok (putem sa trunchiem la 10G, desi datele in sine ocupa 100MB)

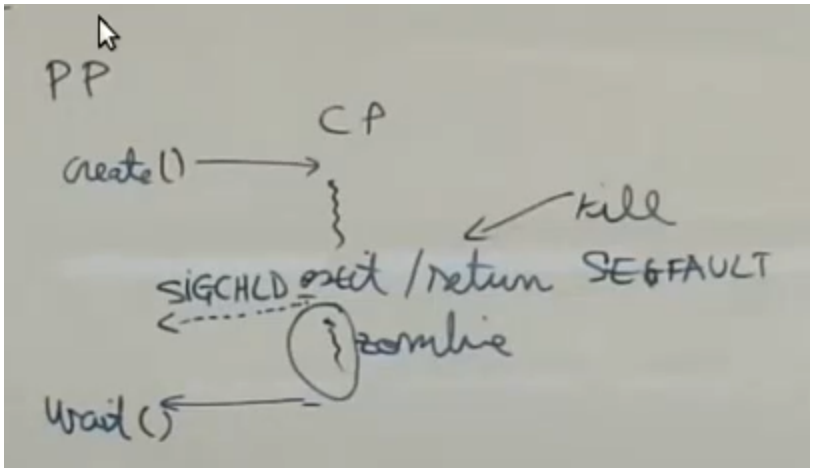
▼ Curs 4 - Procese

- Prefetch → se citeste pe blocuri din memorie (de ex e 512 octeti) desi tu vrei sa citesti doar 256 → optiune de performanta (daca dai read de 256 octeti, vom observa ca offsetul este 512 pt ca se citeste pe blocuri)
- De ce exista ideea de proces? Pentru ca avem mai multe nevoi, lucruri pe care le fac cu un hardware(set de calcule). Atunci cand am nevoie de mai multe aplicatii sau de mai multe instante ale aceleasi aplicatii, am nevoie de procese.
- Proces = program in executie
- Memoria se partitioneaza pt a oferi memorie fiecarui proces
- CPU → se time sharing(scheduling) = se da o cuanta de timp fiecarui proces
- IO → depinde
- Cum facem partitionarea accesului la disk? (mai multe procese vor sa scrie pe disk) → fiecare proces trimite pachete care sunt puse intr-o coada de cereri
- Folosim procese pentru prelucrarea datelor de la input si le trimite la output
- ZONA DE COD → EXECUTABIL → ACTIUNILE REALIZATE DE PROCESE
- Ce interfata ne ofera sistemul de operare pentru a folosi procese? → handle(pid), operatii pe care sa le folosesc pe procese. Pid-ul se obtine prin crearea procesului, iar operatia inversa crearii se obtine prin exit, kill.
- Operatii pe procese: create, kill, suspend, resume, ipc (Inter-Process Communication → mecanism de comunicare si sincronizare intre procese), actiuni de status(monitor, control), wait (asteapta dupa un alt proces)
- Ce argument primeste apelul de creare a unui proces? Numele unui program executabil. Ex: create(executabil)→ intoarce un handle(pid)
- Moduri in care se inchide un proces: cand isi termina executia de la sine, il inchide so (are ceva abnormal gen segfault), il inchide un alt proces (cand userul inchide app, bash-ul trimite un semnal catre proces si este omorat)

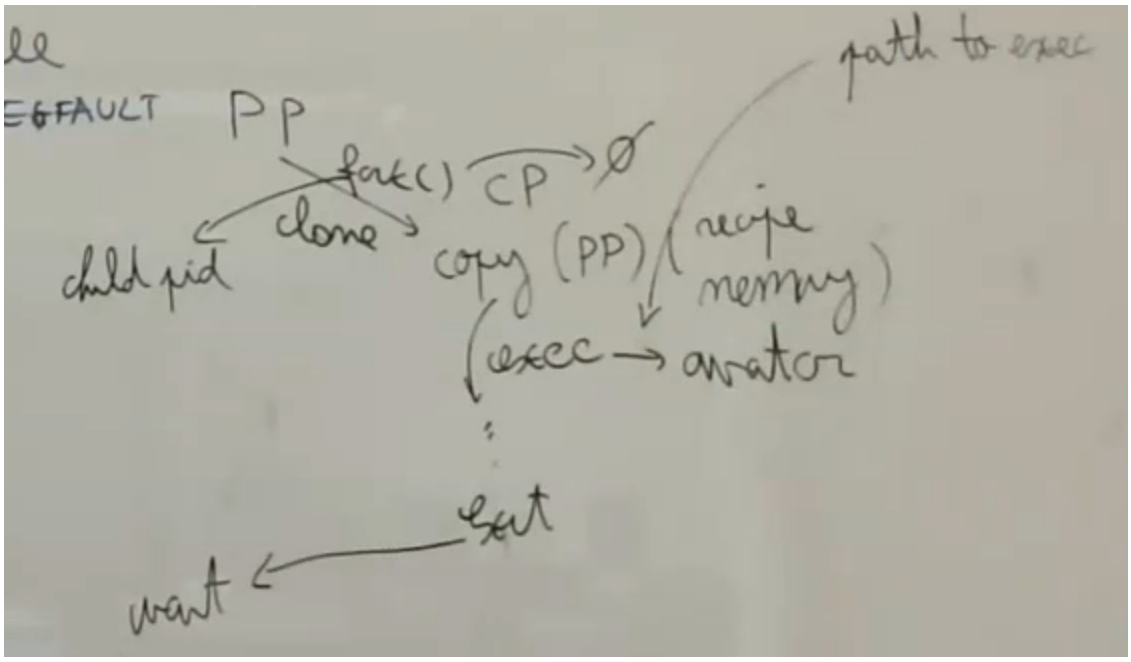
- Cine invoca sistemul de operare sa creeze un proces? Un alt proces (proces parinte) ia executabilul → se invoca loaderul → invoca so → creeaza child process
- Ce se intampla cand procesul parinte moare?
  - Omori procesele copil sau raman orfane si sunt adoptate de procesul init
- Ce se intampla daca un proces moare? Sunt asteptate(wait) de catre procesul parinte (daca nu, devine zombie) si se elibereaza resursele utilizate de acesta. Daca un proces zombie devine orfan, este adoptat de init si omorat

▼ Curs 5 - Procese. Planificarea executiei

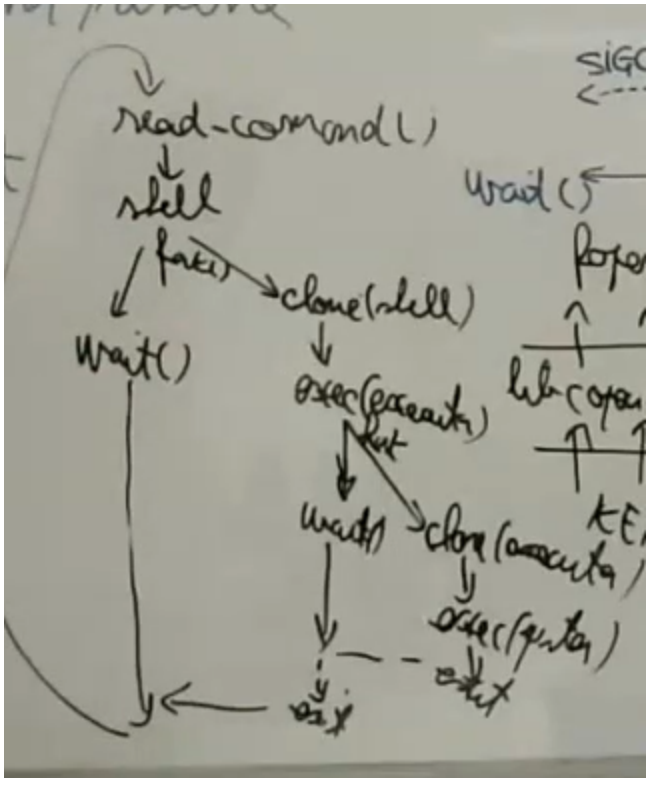
- Apelul wait este realizat pentru a prelua informatiile unui proces copil la sfarsitul executiei acestuia.
- Wait este un apel blocant: parintele asteapta dupa procesul copil sa moara



- fork → cloneaza procesul parinte si intoarce pid-ul procesului copil → prin exec(primeste ca argument calea catre executabil) se inlocuieste vechea “reteta” a parintelui cu un nou program(proces). Se poate realiza exec fara a se face fork inainte, adica nu se realizeaza o copie pe care o transformam, ci se transforma direct procesul curent



Mecanism fork-exec



- Popen creeaza un nou shell(cu fork si exec), care la randul lui face fork si exec pentru a crea unui proces(creeaza un nou spatiu de adrese)
- Imaginea procesului = memoria pe care o ocupa, status, file descriptori

```
pid_t pid;
pid = fork();
pid = fork();
printf("aaa");
```

Se va afisa:

aaa

aaa

aaa

aaa

Deoarece fork cloneaza procesul parinte, iar pana nu se da exec pe procesul copil au acelasi fir de executie. Deci primul fork creeaza o copie a parintelui, iar al doilea fork creeaza cate o copie din fiecare (atat parintele, cat si copia care e identica)

- PCB → process control block

▼ Curs 6 - Planificarea executiei proceselor

- Forkul poate intoarce:
  - -1 → eroare
  - 0 pt procesul copil si pozitiv(pid-ul copilului) pt procesul parinte
- Exec → ori intoarce nimic, ori intoarce eroare
- Pentru a face ca 2 procese copil sa comunice:
  - cream initial pipe-ul deoarece procesele copil il vor mosteni prin tabela de file descriptori
  - facem un fork → proces copil care copiaza VAS (virtual address space)
  - facem alt fork → analog
  - se face operatia dup2(3,0) → reader (3 intrarea in pipe, 0 stdin)

- se face operatia dup2(4,1) → wirter (4 iesirea din pipe, 1 stdout)

  in momentul de fata, la intrarea in pipe pot scrie 3 procese (cele 2 clonate si parintele) si pot citit to 3 intrucat intrarile ocupate din file descriptor table sunt doar 0 (de unde se citeste) si 1 (unde se scrie).

- pentru a face ca pipe-ul sa fie un canal de comunicare doar intre cele 2 procese copil, procesul parinte va face close(3,4)

> 💡 DUP2 → duplica un file descriptor ca in dup(3,0), pune peste intrarea din pipe stdin si dup(4,1) pune peste iesirea pipe-ului stdout. Astfel, rolul pipe-ului devine citirea de la stdin si printarea la stdout.

- 2 procese trebuie sa fie inrudite ca sa poata mosteni acelasi pipe si sa comunice
- Astfel, cele 2 procese au fd table diferite dar acelasi continut care refera catre aceeasi zona de memorie ⇒ pot exista probleme de securitate atunci cand se creeaza mai multe procese copil si au acces la aceelasi zone de memorie(prin mostenirea fd table) ca celalalt proces mare. !man in the middle

- atomic function → functii care chiar daca sunt intrerupte sau mai multe threaduri incearca sa o acceseze simultan acestea isi vor executa operatiile in intregime sau deloc. ⇒ asigura integritatea datelor

Planificarea executiei

→ P1, P3, P3 concura pentru accesul la resursele hardware: memorie (partitioning, isolating), cpu (scheduling), IO (tot scheduling)

- Daca avem un proces si 3 procesoare mai avem nevoie de scheduling? Da, deoarece exista threaduri. (planificam instante de excutie)
- Introducem process state: starea READY (sunt gata sa ma execut), starea efectiva in care el ruleaza (RUNNING), starea BLOCKED (care de fapt waiting dar cica sa nu fie confunzant) este starea in care se afla un proces

cand asteapta resurse (read, write daca e umplut bufferul, wait, locurile pe mutexuri)

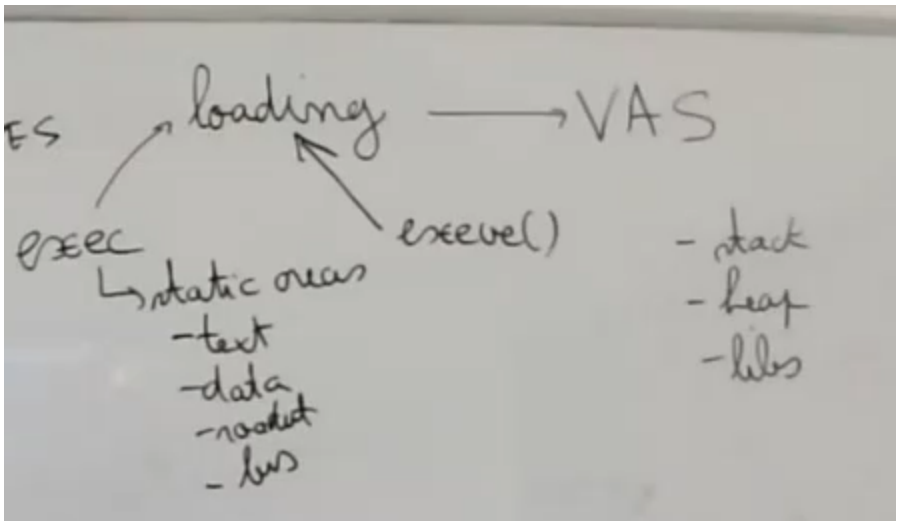
- Cele mai multe procese sunt in starea BLOCKED deoarece asteapta user input.
- Cand este invocat schedulerul, se scoate un proces de pe procesor si se pune altul.
- Cand se apeleaza schedulerul? Si cu cine il inlocuim?
  - Procesul isi incheie executia
  - Procesul se blocheaza → voluntar
  - Exista un proces cu proprietatea mai mare decat altul → involuntar
  - Fuctia yield apelata de un proces ii transmite schedulerului ca vrea sa-si incheie executia → voluntar
  - Procesului i se termina cuanta de timp → involuntar
- In mare parte exista scheduler preemtiv: daca se intampla cv cu un proces, este scos de pe procesor, nu il lasa sa-si termine executia ca la scheduler cooperativ
- Context switch ⇒ schimbarea unui proces cu altul
- Ce procese au prioritatea mai mare, cele IO intesive sau cele CPU intensive? Cele IO deoarece acestea primesc input si se termina rapid.

→ Din blocked in ready cand devine disponibila resursa necesara, din ready in running cand se apeleaza schedulerul (este loc pe procesor si are prioritatea cea mai mare) si din running in ready cand isi da yield, cand expira cuanta de timp si cand are o prioritate mai mica decat altul

▼ Curs 7 - Gestiunea memoriei

- Context switch → cand un proces este inlocuit de altul. Are drept dezavantaj overhead-ul ⇒ ne dorim sa fie rapida si cat mai rara. Ca sa fie cat mai rapida trebuie sa selectam rapid care proces/thread threbuie sa ajunga pe procesor (priority queue). Ca sa fie cat mai rare trebuie sa marim cuanta de timp al procesului.
- Cuanta de timp este stabilita cand trece din ready in running.

- Cuanta de timp mica are avantajul ca ofera interactivitate, simultaneitate. Cuanta de timp mare are avantajul unui overhead mai mic
- Cand un proces iese de pe procesor cu cuanta 0, se trece in ready si cand va trece din nou in running i se va aloca alta cuanta. Daca a iesit de pe procesor cu o cuanta mai mare de 0, i se va realoca abia dupa ce ii expira aceasta.
- Un sistem este incarcat daca avem multe procese in ready
- Waiting time = timpul de asteptare in coada ready
- Daca accesul aplicatiilor la memorie se realizeaza prin partitionale ⇒ 2 neajunsuri legate de lipsa dinamismului deoarece este greu de modificat vechea dimensiune a proceselor si in cazul stergerii unuia din memoria rezulta fragmentarea acesteia (un proces poate sa nu incapa in locul ramas liber) ====⇒ accesul la memorie se realizeaza astfel prin VAS (fiecare proces scrie in propriul sau spatiu de adrese) care este apoi mapata in memoria fizica
- Cu VAS ai mereu aceeasi adresa in memorie
- Ce contine PCB?
  - O lista de TCB
  - FD table
  - VAS ⇒ o lista de zone de memorie, unde o zona este descrisa de adresa de inceput, permisiuni si o dimensiune
- Sigsegv → se realizeaza cand nu ai permisiuni asupra unei zone sau cand se acceseaza o zona nealocata



- Cum se aloca pe stiva? Se scade stack pointerul.

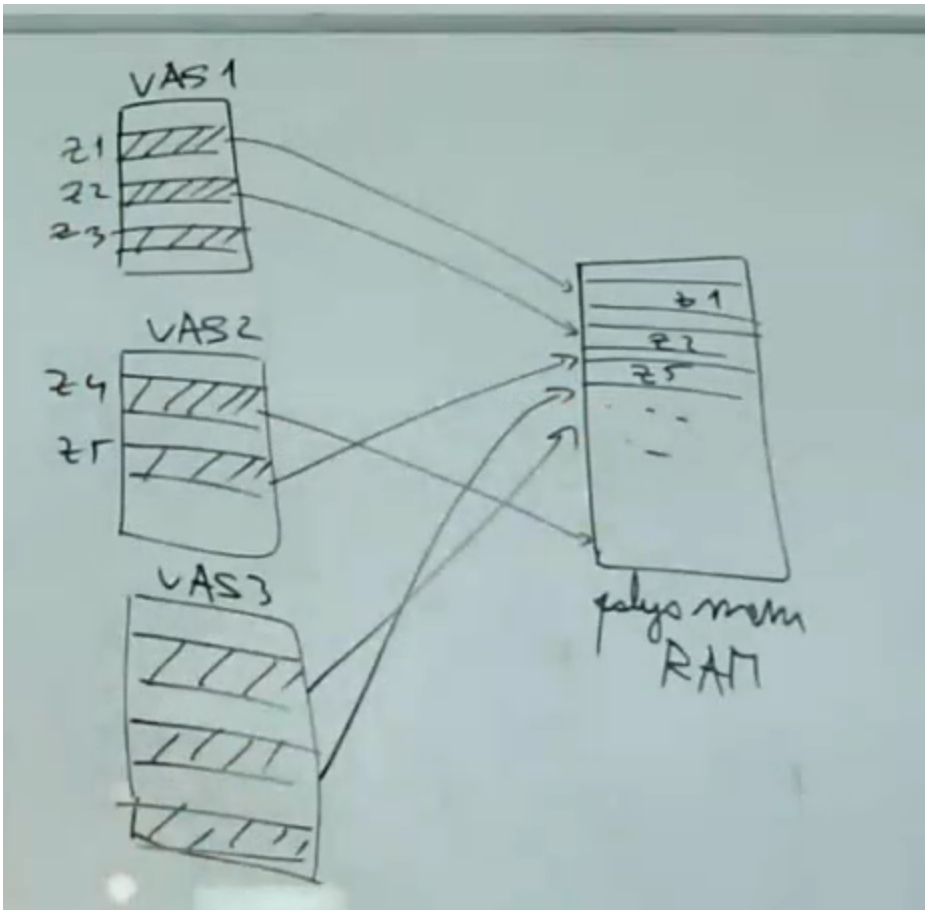
▼ Curs 8 - Gestiunea memoriei. Memoria virtuală

- Numar threaduri - cel putin egal cu numarul de procese
- Executabilul populeaza spatiul virtual de adrese → se regasesc zonele statice: text, data, rodata, bss
- Zonele dinamice sunt populate pe masura ce sunt apelate functii: stack si heap. Zona dinamica mai e si libs.
- VAS se actualizeaza cand: se aloca o zona noua (prin mmap), se extinde o zona (malloc, mmap, stack use), se da update la content (memory write) in zonele stack, heap, data, bss, se da update la permisiuni (mprotect)
- Functia strcpy provoaca update de content si poate provoca extinderea zonei de memorie (se scrie intr-un buffer si se depaseste memoria alocata pe stiva ⇒ se aloca automat cand se depaseste limita stivei)
- Functia strdup (aloca memorie pt un sir si copiaza un sir peste el) va provoca extinderea heapului.
- MTU - memory translation unit. La nivelul sistemului de operare, memoria virtuala este continua, pe cand memoria fizica asociata zonelor din vas poate fi fragmenata
- Cate zone text are un proces? 1 + nr de biblioteci dinamice
- Cate stive sunt in vas? Cate una pe fir de executie(thread)

- Pe heap se prealloca spatiu ⇒ malloc va genera un apel de sistem doar atunci cand nu mai are memorie din cea prealocata. Se prealloca ca sa minimizam tranzitia dintre user space si kernel space
- Apelul de sistem brk extinde zona de heap preallocata, insa apelul mmap lucreaza pe o zona noua.
- Mallocul cand are dimensiuni mici lucreaza cu heapul prin apeluri de sistem brk. Insa mallocul poate sa apeleze si mmap care lucreaza cu alte zone (cand se depaseste limita de memorie pe care o poate alloca malloc)

▼ Curs 9 - Memoria virtuala

- mapare → asociere, corespondenta unei zone de memorie virtuala cu o zona de memorie fizica(”mapam zona virtuala peste memoria fizica”)



- Diferenta dintre spatiul fizic si spatiul virtual: un singur spatiu fizic, dar mai multe vas (spatiul virtual este de dimenisune mai mare ⇒ exista cazuri

cand nu se pot mapa toate informatiile din vsa peste memoria fizica)

- Tabela de mapare<KEY, VAL> → pid virtual zone, phys zone> (in teorie, insa in fractica nu avem nevoie de asid deoarece se creeaza o tabela per proces)
  - pid = ASID = address space id
- Se creeaza o tabela de mapare per proces (page table) si are atatea intrari cate zone de memorie are.
- Fiecare zona de memorie este o zona continua de pagini.
- O pagina are 4K.
- Tabela de mapare = page table
- Pagini virtuale = virtual pages, iar paginile fizice = frames
- Tabela de pagini are 2^20 intrari si contine un entry pentru orice pagina, fie ea valida sau nu

```
int a[10];
a[20] = 0;
// nu voi primi segfault deoarece
// pentru a se va aloca o pagina intreaga,
//iar a[20] nu o depaseste

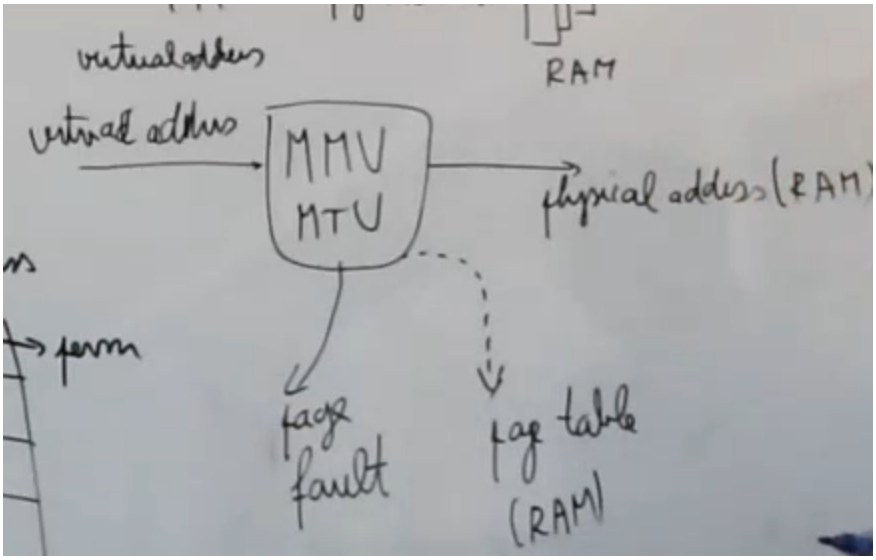
// daca se foloseste un index care sa treaca
// de dimensiunea paginii, va intoarce seg (asta daca
// nu si urmatoarea pagina este valida)
```

- Pentru a afla adresa fizica, se ia offestul din pagina virtuala, se afla frame-ul aferent paginii din tabela de pagini la care se adauga offestul ⇒ adresa fizica
- O intrare in tabela retine id-ul frame-ului, permisiuni, bit care retine daca pagina este valida sau invalid
  - daca accesezi o pagina invalida, primesti o exceptie care se numeste page fault
  - daca dorim sa facem o operatie pentru care nu avem permisiuni ⇒ page fault

- Tabela de pagini se afla tot in memorie ⇒ se fac 2 accese in memorie
- TLB, un cache penru intrari in tabela de pagini (utila pentru a retine tabela de pagini in cache si a nu face acces la memorie de prea multe ori)
- PTBR → refera pentru fiecare address space adresa din memoria fizica unde se gaseste tabela de pagini

▼ Curs 10 - Memoria virtuala. Analiza executabilelor si proceselor

- TLB = translation lookaside buffers (memoria cache) care este aproape de procesor si retine informatiile despre cele mai accesate pagini fizice. (chacheuieste tabela de pagini)



- MMU - memory management unit - parte din procesor(primeste adresa virtuala si scoate adresa fizica accesand tabela de pagini ale procesului curent). Daca pagina virtuala nu are corespondent in memoria fizica (este invalida) sau nu are permisiuni de executie se genereaza page fault
- PTBR - page table base register care retine pentru fiecare VAS unde este tabela de pagini asociata din spatiul fizic
- TLB - se foloseste pentru a reduce din overheadul temporal
- Paginarea ierahica rezolva problema overheadului spatial

PAGEFAULT - cand se acceseaza o pagina invalida sau cand nu avem permisiuni

- page fault - cauzeaza o intreupere software?
- page fault handler - exista 2 posibilitati
  - segfault
  - other →
    - map/remap
    - protect (cu noi permisiuni)
- mmap → rezerva spatiu virtual (se adauga in vas ca fiind pagini invalide)
- on demand paging → se aloca spatiu in memoria fizica atunci cand se acceseaza spatiul virtual rezervat cu mmap si se mapeaza peste memoria virtuala

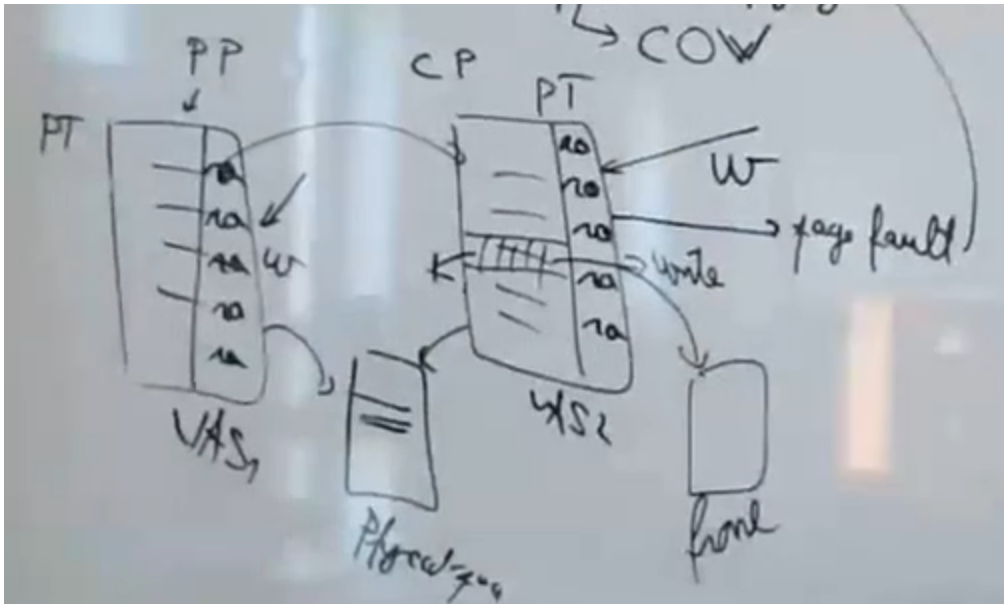
PAGEFAULT HANDELR PASI:

1. s-a rezervat spatiu virtual (nu modifica tabela de pagini)
2. se face acces catre o zona valida din punct de vedere al spatiului virtual (daca ar fi invalida s-ar da segfault)
3. apoi adresa virtuala intra in MMU care consulta tabela de pagini si observa ca este nevalida si se genereaza pagefault ⇒ pagefault handler
4. handler-ul observa ca este o zona virtuala valida si folosind PTBR-ul se merge la tabela de pagini asociata si se asociaza un frame fizic, se modifica ca valid si i se schimba permisiunile in permisiunile zonei.
5. se reia instructiunea care a generat pagefault, se observa ca acum este valid, are PFN (page frame number)

- Entitatea care se ocupa de modificarea tabelei de pagini pentru a o face valida si a-i asocia un frame este pagefault handler.

- Cand un pagefault nu cauzeaza segfault? Accesez o intrare marcata invalida, dar in VAS este intr-o zona valida.
    - o intrare poate fi invalida daca: s-a facut on demand paging si s-a marit vas sau prin swap - entitate pe disk (nu mai e loc in memoria fizica si o inlocuiesti cu o alta pagina)

    - Se mai poate genera pagefault atunci cand se face fork dintr-un proces parinte si procesul copil va avea un VAS diferit dar cu continut identic ⇒ vor pointa catre aceeasi zona din spatiul fizic, iar permisiunile vor fi doar read. Cand procesul copil incearca sa scrie in tabela ⇒ pagefault cauzat de lipsa de permisiuni de write ⇒ copy on write



- copy-on-write este modalitatea prin care procesele copil primesc corespondența pagini - frames. Inițial, spațiul virtual de adrese al procesului copil point-ează către același spațiu fizic de adrese. Permisiunile sunt schimbate pe read only, și de abia la scriere, se duplică frame-urile, iar permisiunile sunt updatate.

- shared memory - 2 sau mai multe procese pot imparti acelasi PFN.
- bibliotecile dinamice - shared objects = sunt folosite ca sa nu pastram cate o biblioteca pentru fiecare VAS.

▼ Curs 11 - Analiza executabilelor și proceselor. Securitatea memoriei

- PTBR = reprezinta adresa fizica catre tabela de pagini a procesului curent. Atunci cand procesul curent este inlocuit de un altul(schimbare de context) se schimba valoarea PTBR-ului.

NO MORE

▼ Curs 12 - Fire de executie

- thread = execution unit
- event base (tranzitii intre evenimente)> thread base (thread pt fiecare veniment), event base este mai rapida insa mai greu de programat
- Hardware execution units = CPU, GPU, FPGA ⇒ multithreading sau multiprocess, imaprtindu-se pe mai multe unitati de executie
- avantaj multithread → threadurile partajeaza memoria → comunicare mai facila (overhead redus de comunicare)
- dezavantaj multithread → pot comunica pt ca ele paseaza memoria ⇒ apar probleme de sincronizare a datelor, un thread poate modifica datele altuia
- context switch intre threadurile aceluaisi proces → reducerea overheadului deoarece nu mai trb sa se schimbe spatiul de adrese, ptbr… si pt ca threadurile sunt in acelasi address space
- Ce interfata avem pt lucrul cu procese? → functii: create, wait, notify, status, control (modificarea prioritatii, CPU affinity)
- Ce interfata avem pt lucrul cu threaduri? → functii: create (nu mai e nevoie sa se dea load ca in cazul procesului cand se dadea load la executabil → primeste ca argument pointerul catre o functie), wait, notify
  - create thread → intoarce un TID cu care obtinem TCB
  - fiecare thread are o stiva necesara pentru flowul curent de executie (stiva dezvaluie istoriul, ce am fct pana atunci)

        - instruction pointerul → what to do
        - stack pointerul → ce-am fct
    - Numarul minim pe threaduri per proces = 1, iar numarul maxim: 2^32 = 3G VAS / 8M (fiecare stiva ale unui thread) = NUM_MAX_THREADS
    - Daca se face context switch intre threaduri ce apartin a 2 procese diferite: se schimba VAS, PTBR, se da flush la TLB

▼ Curs 13 - Fire de execuție. Sincronizare

- 2 threaduri au stive diferite, dar partajeaza acelasi spatiu de memorie(ale aceluiasi proces)
- stiva nu este izolata de celalte threaduri
- pot exista variabile globale, dar speifice fiecarui thread “static __thread var” → THREAD LOCAL STORAGE
- Threadul principal are TLS-ul alocat pe heap

Schimbare de context threaduri

- se scoate un thread (se salveaza registers) de pe procesor si se pune altul in locul lui (restauram registers) → cand se schimba threadurile aceluiasi proces
- daca nu fac parte din acelasi proces trb sa comutam spatiul de adrese ⇒ salvam PTBR, restauram PTBR ⇒ flush la TLB
- green threads = threaduri care sunt facute doar in userspace⇒ avantaj: avem control ajupra loc, plaificarea se face fara a fi nevoie de kernel, dar au drept dezavantaj faptul ca nu sunt vazute de so

▼ Curs 14 - Sincronizare

- ITC (inter thread communication) mai rapid decat IPC deoarece threadurile au shared memory implicit(partajeaza intreg spatiul de adrese al unui proces). Se poate face shared memory si la procese dar trebuie sa realizam mecanismul de comunicare.

PRIMITIVE DE SINCRONIZARE

- Un singur thread poate accesta o zona critica la un moment dat
- Primitive: lock si unlock
- Mecanismele care au primitive de lock si unlock: mutex si spinlock. (acces exclusiv)
- ordonare: exista un nivel de dependenta intre threaduri unde un thread poate sa porneasca doar dupa ce s-au terminat cele dinaintea lui
    - primitivele mecanismelor de ordonare: wait si notify ⇒ garantez o anumita secventiere a operatiilor
    - mecanisme care au primitive wait si notify: semafore, variabile conditie
- race condition: o situatie in care executia unui program depinde de modul in care se executa threadurile. (timpul la care se executa)
- deadlock = o folosire necorespunzatoare a primitivelor de lock = 2 procese sunt in blocked, fiecare asteptand resurse de la cealalta sau se face dublu lock pe un proces
- mutexul → 2 campuri: aquired (0 sau 1, daca este luat sau nu) si o lista de threaduri
- un mutex contine un spinlock
- spinloc - un intreg
- busy waiting = tehnica prin care procesul verifica constant daca o conditie se intampla si executa urmatoarele instructiuni doar daca aceasta este indeplinita(threadul asteapta pana cand vede ca este singurul care a modificat variabilele si fluxul executiei si apoi isi termina executia ⇒ securitate dar produce overhead)

▼ Curs 15 - Dispozitive de intrare-iesire

- Ce primitiva de sincronizare are in implementarea functiei lock o instructiune de tip atomic_cmpxchg? → spinlock
    - atomic_cmpxchg(&var, 0, 1) ⇒ intoarce var

    - if(var == 0) var = 1
        - return 0;
        - return 1;

- In ce situatie nu este atomica add[ebp - 32], eax? Pe un sistem multicore! ⇒ se poate sa fiecare threads sa incerce sa scrie la acea adresa ⇒ trebuie prefixata folosind lock(inchide magistrala, doar corul curent are acces) ⇒ limitare hardware

COMPONENTE IO

- process view → IO channel care poate fi ori catre alte procese (canale de timp socket), ori catre deviceuri (canale de timp fisier)(keyboard, monitor, disk, placa de retea)
- system view → doar device-urile sunt vazute ca IO uri
- interfata de socket are placa de retea cu care comunica un proces → nu este nici sursa, nici destinatie de informatie
- pipe → buffer in care un proces scrie si altul citeste
- De ce e IO important? Este partea critica in performanta aplicatiei.
- Driverele ocupa cea mai mare parte din codul nucleului ⇒ pentru ca ?
- ISA(interfata hardware software) expune instructiuni, registrele procesorului, modul de adresare
- Deviceurile au niste buffere hardware care retin informatia care sa mearga ls software ⇒ le accesez printr-o magistrala si avem nevoie de un mod de adresare
    - instrcutiuni dedicate + adrese ⇒ putem accesa registers ale deviceurilor

- PMIO → i/o address space (in - intr un port(adresa unui registru) , out - intr-un port)
- MMIO - RAM sau IO device
- device virtual = nu are forma de adresare (nu creeaza intreruperi) - nu are controller si device fizic ⇒ va comunica direct cu memoria sau cu un alt proces care simuleaza device-ul fizic
- device virtual = /dev/null ⇒ citeste un buffer si nu face nimic cu ele